

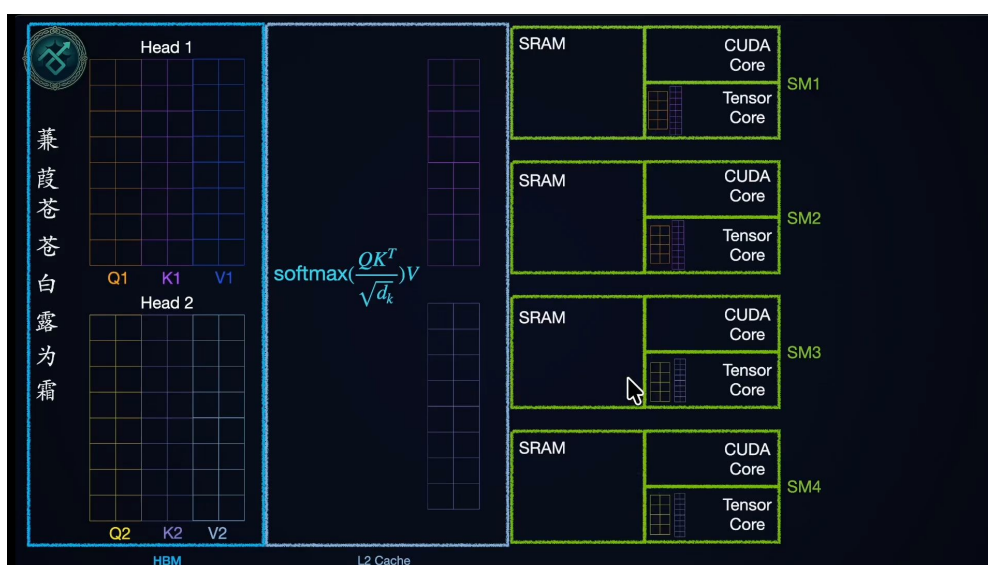
课程笔记

GPU 如何一步步计算自注意力机制

从 GPU 存储层级，到自注意力的硬件级计算全流程

lecture-to-notes 自动讲义

2026 年 6 月 20 日



视频作者/频道：向量隐修会

发布平台：Bilibili

视频时长：约 20 分钟

视频链接：<https://www.bilibili.com/video/BV1zFFTziEdL>

目录

1 引言：换一个视角看自注意力	2
1.1 本章小结	2
2 GPU 的内部结构：计算单元与存储层级	2
2.1 从 H100 到一个基本单元 SM	2
2.2 存储层级：越往里越小、越快	3
2.3 数据如何进出 GPU：PCIe 与 NVLink	4
2.4 简化模型	5
2.5 本章小结	6
3 自注意力的输入准备：Embedding 与参数矩阵	6
3.1 从文字到嵌入矩阵	6
3.2 本章小结	7
4 第一步：计算 QKV 矩阵	7
4.1 本章小结	7
5 第二步：注意力分数 QK^T、缩放与 Softmax	8
5.1 QK^T ：多头 $\times$ 多 SM 的双重并行	8
5.2 缩放 $/\sqrt{d_k}$ 与带“减最大值”的 Softmax	8
5.3 本章小结	9
6 第三步：加权求和 $\times V$ 与多头混合	10
6.1 $P \times V$	10
6.2 多头混合：输出投影 W_O	10
6.3 自注意力计算流程总览	10
6.4 本章小结	11
7 总结与延伸：藏在流程里的“内存墙”	11
7.1 作者的收尾：问题出在哪	11
7.2 笔者的延伸提炼	11
7.3 拓展阅读	12

1 引言：换一个视角看自注意力

大多数人理解的自注意力，停留在 $\text{softmax}(QK^T/\sqrt{d_k})V$ 这条公式。但当这条公式真正跑在 GPU 上时，它被拆成了一连串**矩阵搬运** + **矩阵乘法**的硬件操作：数据在 HBM、L2、SRAM 之间反复流动，被切分给多个计算单元并行处理。本讲就从 GPU 的内部结构出发，以一句诗”**蒹葭苍苍，白露为霜**”（8 个字、2 个注意力头）为例，一步一步看自注意力是如何被算出来的。

理解这套硬件级流程，不仅能让你真正”看见”注意力，也是理解下一讲 **FlashAttention**（以及 KV Cache、PagedAttention 等推理优化）的必要前提——因为这些优化的全部动机，都藏在本讲最后揭示的那个”内存墙”问题里。

1.1 本章小结

我们将分两大块展开：先建立 GPU 的**存储层级与计算单元**心智模型，再把自注意力的每一步落到这套硬件上执行。

2 GPU 的内部结构：计算单元与存储层级

2.1 从 H100 到一个基本单元 SM

打开一张 H100 的内部结构图，密密麻麻看似复杂，但其实它由大量相同的基本计算单元堆叠而成。这个基本单元叫 **SM (Streaming Multiprocessor, 流式多处理器)**。

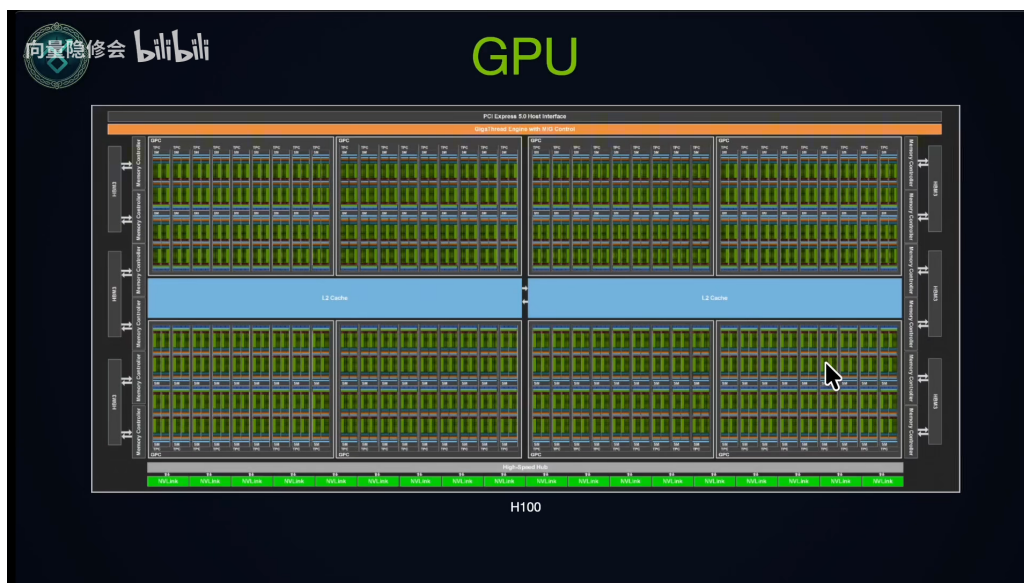


图 1: NVIDIA H100 的整体布局：大量绿色的 SM 阵列，中间蓝色横贯的是 L2 Cache，底部是 HBM 接口。¹

一个 SM 内部由 4 个 SM 分区构成，每个分区都包含同样的四种部件：

¹视频画面时间区间：00:00:06-00:00:12。



图 2: SM 分区内部: 寄存器堆 Register File (16,384×32-bit)、INT32/FP32/FP64 的 CUDA Core、第 4 代 Tensor Core、以及特殊功能单元 SFU。²

SM 分区的四种部件

- **寄存器堆 (Register File):** 离计算核心最近的存储, 为计算提供数据、也接收计算结果;
- **CUDA Core:** 负责基本的加减乘除等标量运算;
- **Tensor Core:** 专门负责矩阵乘法 (深度学习的算力主力);
- **SFU (Special Function Unit):** 负责指数、对数等特殊函数运算 (如 softmax 里的 e^x)。

除 4 个分区外, SM 内还有一块 **L1 Data Cache / Shared Memory**: 二者共享同一块物理空间, 可按比例划分谁多谁少; 其中 Shared Memory 用于 4 个分区之间共享数据。为简化后续讨论, 本讲把 **寄存器堆 + L1/Shared Memory** 合起来统称为 **SRAM**。

2.2 存储层级: 越往里越小、越快

把视角拉到整张 GPU (简化为只有 4 个 SM)。每张 GPU 都有自己的显存 **HBM (High Bandwidth Memory)**, 可以理解成一个“数据大仓库”——训练或推理时数据都要先存在 HBM 里。关键在于这套存储的 **层级与速度**:

²视频画面时间区间: 00:00:51–00:01:13。

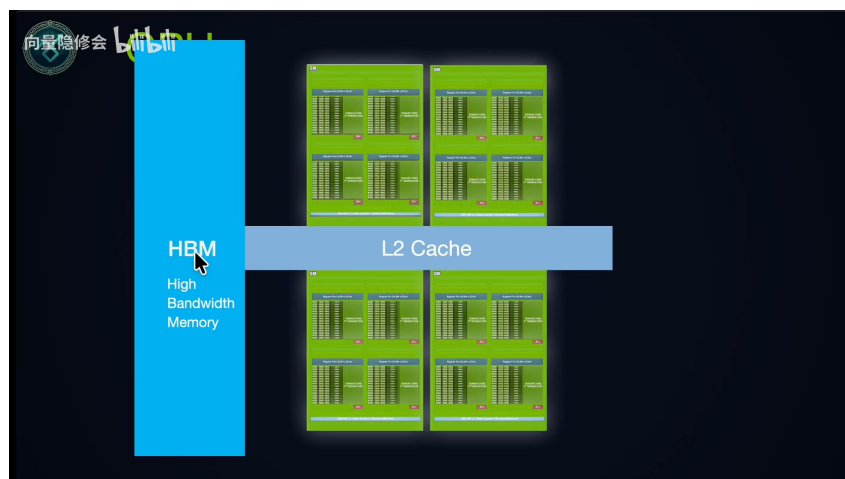


图 3: GPU 存储层级: 左侧 HBM 是最大的仓库, 经中间的 L2 Cache, 数据才能进入各个 SM 内部。SM 与 HBM 之间不能直接交换数据, 必须途经 L2。³

数据的完整流向是: HBM → L2 Cache → SRAM(L1/Shared) → 寄存器 → 计算核心, 算完再原路写回。下面用一张图把这套层级的”容量 vs. 速度”反比关系画清楚:

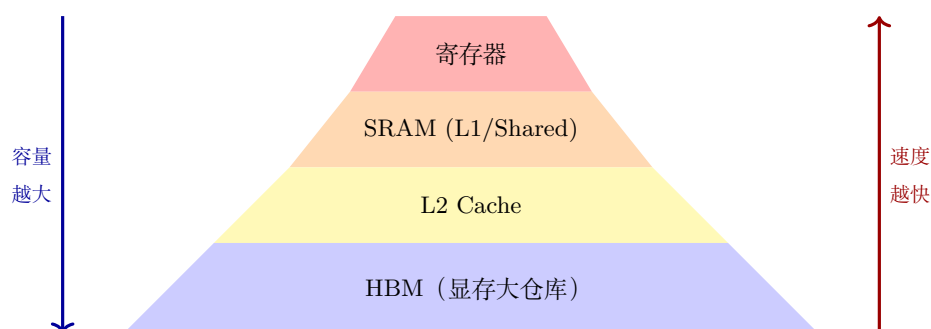


图 4: GPU 存储层级金字塔 (笔记自绘): 越靠近计算核心 (顶部), 容量越小但速度越快; 越往外 (底部 HBM), 容量越大但速度越慢。这个”容量与速度的反比”是后文所有性能问题的根源。

2.3 数据如何进出 GPU: PCIe 与 NVLink

GPU 内部的数据流讲清楚了, 那数据怎么从外部进入 GPU?

³视频画面时间区间: 00:03:30-00:03:46。

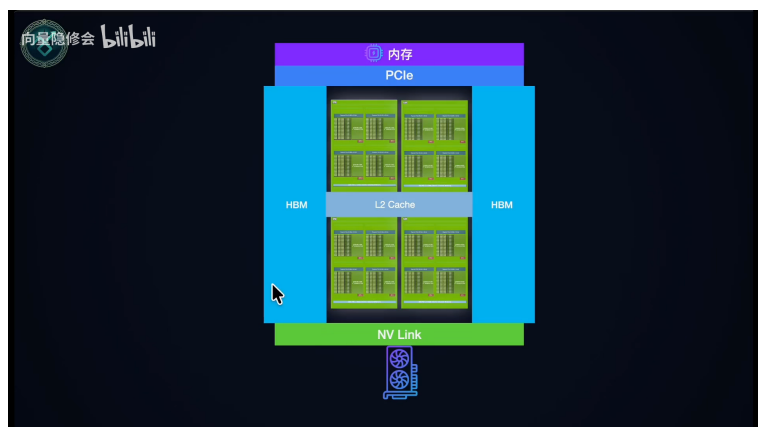


图 5: 两条对外通道: 上方经 **PCIe** 连接 CPU 与内存 (数据路径: 磁盘 → 内存 → CPU → PCIe → HBM); 下方 **NVLink** 直连另一张显卡, 不经 CPU/内存。⁴

PCIe vs. NVLink

PCIe 是 GPU 与 CPU/内存之间的通道: 数据通常先从磁盘到内存, 再由 CPU 经 PCIe 送进 HBM。

NVLink (英伟达显卡特有) 直接连接两张显卡, 中间不经过 *CPU* 和内存, 速度远高于 PCIe。当训练/推理的大模型单卡显存放不下时, 就用 NVLink 把多张显卡连在一起协同。

2.4 简化模型

后续讨论基于一个简化后的 GPU: 每个 SM 看作 **SRAM + CUDA Core + Tensor Core (+ SFU)**, 外接 HBM、L2, 对外有 PCIe / NVLink。

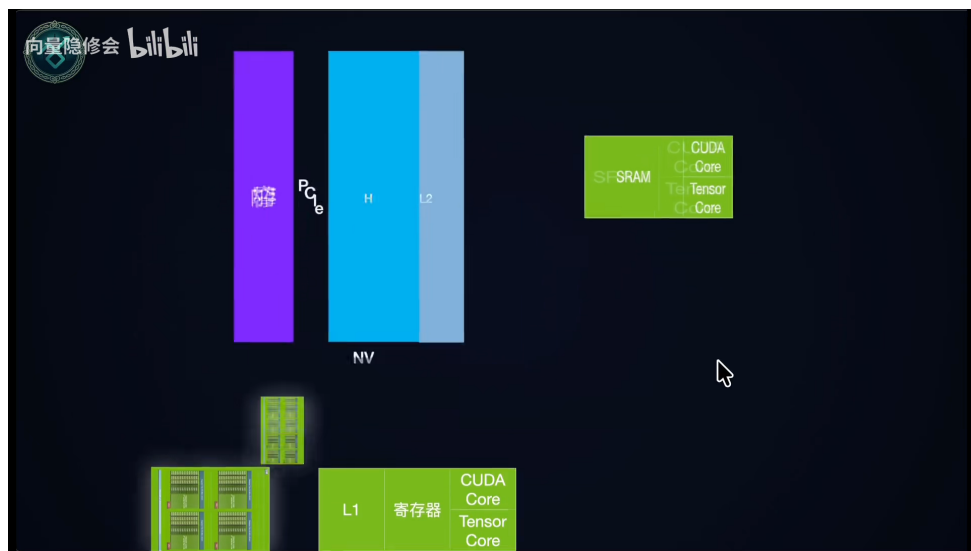


图 6: 本讲使用的简化 GPU 模型: 每个 SM 抽象为“SRAM + 计算核心”, 配合 HBM / L2 与 PCIe / NVLink 通道。⁵

⁴视频画面时间区间: 00:04:04–00:04:46。

⁵视频画面时间区间: 00:05:16–00:05:45。

2.5 本章小结

记住两条主线：计算由 CUDA Core（标量）、Tensor Core（矩阵）、SFU（特殊函数）分工完成；存储是 HBM→L2→SRAM→寄存器的层级，容量递减、速度递增，且 SM 与 HBM 必须经 L2 中转。这两条线决定了自注意力在 GPU 上的全部执行方式。

3 自注意力的输入准备：Embedding 与参数矩阵

3.1 从文字到嵌入矩阵

以“蒹葭苍苍，白露为霜”为例。每个字是一个 **token**，经 tokenizer 得到一个数字编号（如图中 9, 12, 78, ...）。再用一张**嵌入矩阵（Embedding）**——它的每一行就是一个 token 的向量——按编号去查表，就把这串文字变成了一个数据矩阵 X 。

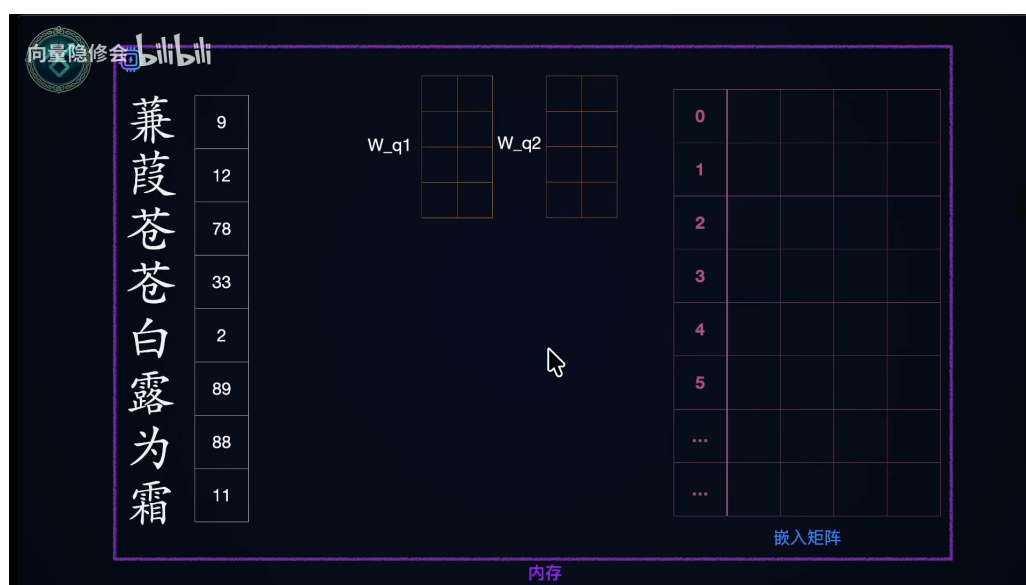


图 7: 输入准备：8 个字各有 token 编号，通过嵌入矩阵按编号查表得到行向量；右侧即嵌入矩阵，左中是 W_{q1}, W_{q2} 等参数矩阵。⁶

本例的维度约定（务必记住）

- 序列长度 8（8 个字），**注意力头数 2**，每头嵌入维度 2；
- 嵌入后的数据矩阵 X 形状为 8×4 （ $4 = 2 \text{ 头} \times 2 \text{ 维}$ ）；
- 参数矩阵 W_Q, W_K, W_V 两头拼在一起，合成大参数矩阵 W_{QKV} ，形状 4×12 （ $12 = 3(QKV) \times 2 \text{ 头} \times 2 \text{ 维}$ ）。存储时它们本就是拼在一起、一起加载的。

参数矩阵和数据下载到内存后，经 PCIe 转移到 HBM，计算才正式开始。

⁶视频画面时间区间：00:05:54–00:07:10。

3.2 本章小结

输入被转成 8×4 的 X 矩阵，参数被组织成 4×12 的 W_{QKV} ，全部就位于 HBM。接下来的每一步，都是”把矩阵从 HBM 搬进 SM、用 Tensor Core 算、再写回 HBM”的循环。

4 第一步：计算 QKV 矩阵

有了 X (8×4) 和 W_{QKV} (4×12)，QKV 就是一次矩阵乘法 $X W_{QKV}$ ，结果是 8×12 。GPU 怎么并行？把 8 个字按行切成 4 份（每份 2 个字），分给 4 个 SM：

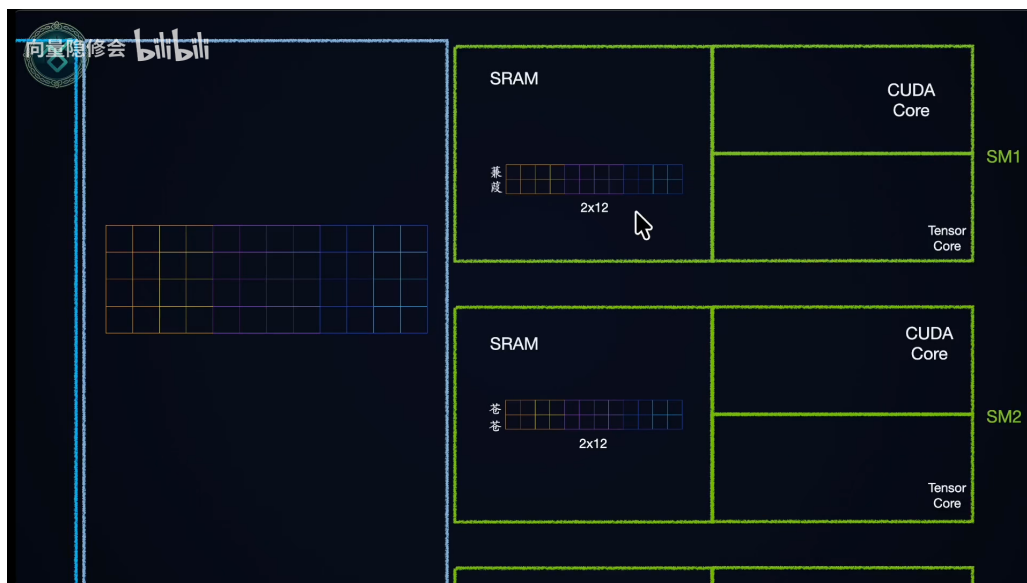


图 8: 4 个 SM 分工算 QKV：每个 SM 拿到 2×4 的 X 子块，乘以 4×12 的 W_{QKV} ，用 Tensor Core 算出 2×12 的结果块；4 块合起来就是 8×12 的 QKV 大矩阵。⁷

一个关键优化：参数只加载一次

4 个 SM 都需要同一个 W_{QKV} 大参数矩阵。所以只需把它从 HBM 加载到 L2 Cache 一次，4 个 SM 各自从 L2 读取即可，不必从 HBM 重复读 4 次。哪个 SM 先算完，就先把结果写回 HBM。这种”把共享数据留在更快的缓存里、避免重复访问 HBM”的思想，正是后续一切优化的母题。

算出 8×12 的 QKV 大矩阵后，按列切开： $Q_1 Q_2 / K_1 K_2 / V_1 V_2$ 分别对应两个头，重新组合成两套 QKV（第一头一套、第二头一套），每个矩阵形状 8×2 。

4.1 本章小结

QKV 计算 = 一次 $8 \times 4 \cdot 4 \times 12 = 8 \times 12$ 的矩阵乘，按行分给 4 个 SM 并行，参数经 L2 共享只读一次。切头后得到两套 8×2 的 Q, K, V 。

⁷ 视频画面时间区间：00:09:10–00:09:30。

5 第二步：注意力分数 $QK^\top$ 、缩放与 Softmax

5.1 $QK^\top$ ：多头 $\times$ 多 SM 的双重并行

自注意力公式 $\text{softmax}(QK^\top/\sqrt{d_k})V$ 的第一步是 Q 与 $K^\top$ 相乘。这里有两个层次的并行：2 个头 $\times$ 4 个 SM。

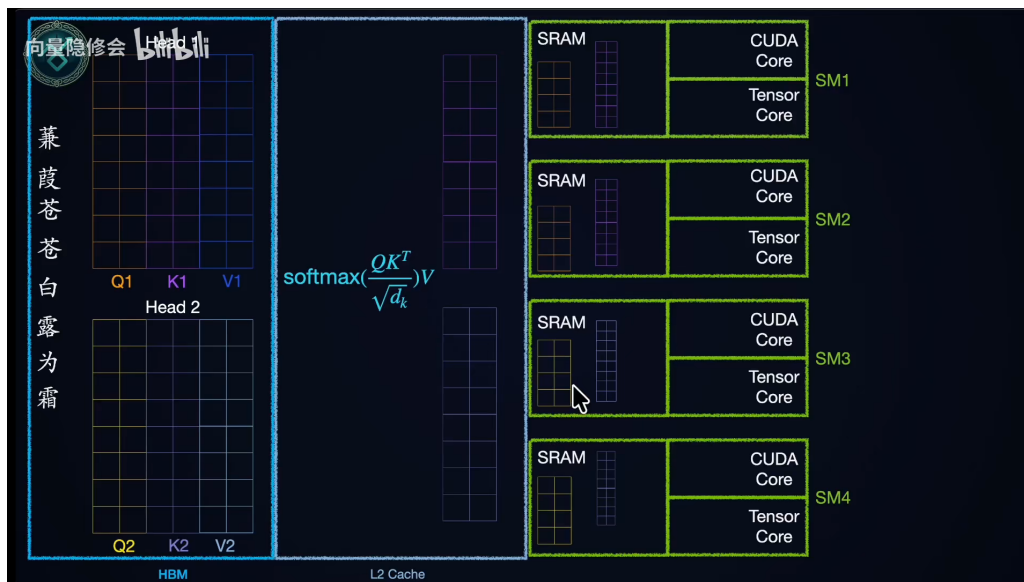


图 9: $QK^\top$ 的硬件布局：HBM 里存着两头的 Q_i, K_i, V_i ；让 SM1+SM2 负责第一头、SM3+SM4 负责第二头；每对 SM 内部再把 8 个字分成两半（各处理 4 个字）。⁸

又一次”避免重复读 HBM”

计算第一头时，SM1 和 SM2 都需要完整的 K_1 矩阵（因为每个 query 要和所有 key 做内积）。所以 K_1 同样只加载到缓存一次，两个 SM 都从缓存读，**不从 HBM 读两次**。每个 SM 算 4×2 的 Q 块乘 2×8 的 $K^\top$ ，得 4×8 ；两个 SM 拼成 8×8 的注意力分数矩阵。

5.2 缩放 $/\sqrt{d_k}$ 与带”减最大值”的 Softmax

得到分数矩阵后，先对每个元素除以 $\sqrt{d_k}$ （一个简单的标量除法，用 CUDA Core 算），得到注意力分数矩阵 S_1, S_2 。接着对每一行做 **Softmax**：

⁸视频画面时间区间：00:11:50–00:12:13。

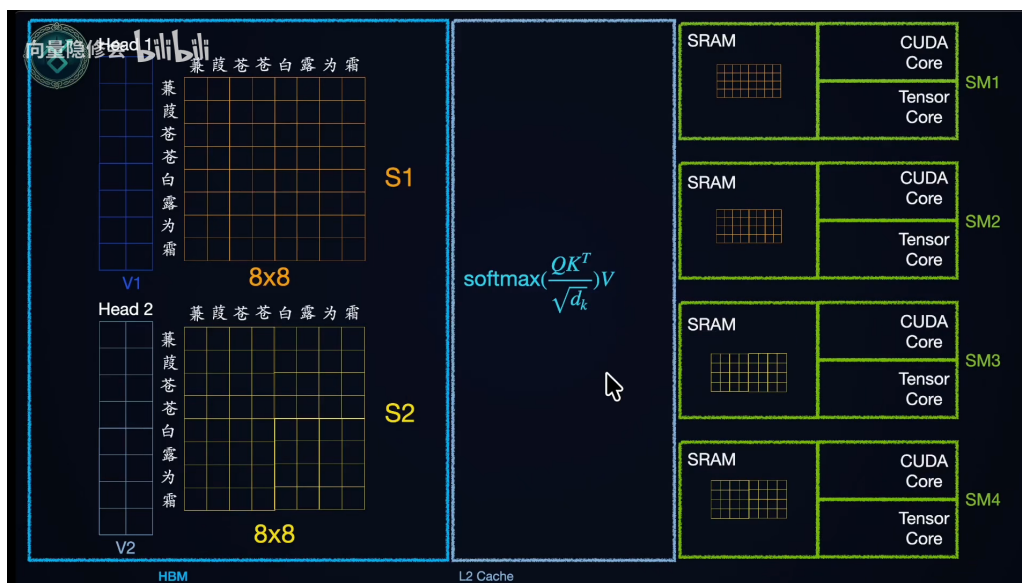


图 10: 两头的注意力分数矩阵 S_1, S_2 (各 8×8)。Softmax 逐行进行; 16 行分给 4 个 SM, 每个 SM 处理 4 行。⁹

视频用的 Softmax 公式与教科书略有不同——分子指数里减去了该行的最大值 m :

$$p_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j.$$

其中:

- x_i : 该行第 i 个注意力分数; m : 该行的最大值;
- 减 m 不改变 Softmax 的结果 (分子分母同乘 e^{-m}), 但能防止数值溢出。

为什么要减最大值——数值溢出

若某个 x_i 很大 (比如 100), e^{100} 会大到浮点数无法表示而溢出。减去该行最大值后, 指数最大不超过 $e^0 = 1$, 数值稳定。这一招的更深原理 (以及它如何让 Softmax 能”分块在线计算”), 正是下一讲 **FlashAttention** 的核心。

计算时, 指数 e^x 用 SFU, 分母求和与最后的除法用 CUDA Core。算完得到注意力权重矩阵 P_1, P_2 。

5.3 本章小结

QK^T (Tensor Core) $\rightarrow$ 缩放 $/\sqrt{d_k}$ (CUDA Core) $\rightarrow$ Softmax (SFU 算指数、CUDA Core 求和除法), 全程多头 $\times$ 多 SM 并行、共享矩阵只读一次。得到权重矩阵 P_1, P_2 。

⁹视频画面时间区间: 00:13:40–00:14:17。

6 第三步：加权求和 $\times V$ 与多头混合

6.1 $P \times V$

Softmax 这一支算完，公式里只剩 V 没用。第三步就是用权重矩阵去乘 V ：仍是两头分开、4 个 SM 并行。第一头需要完整的 V_1 (从缓存读)，每个 SM 算 4×8 的 P 块乘 8×2 的 V ，得 4×2 ；拼起来每个头得到 8×2 ，两头再拼成 8×4 。

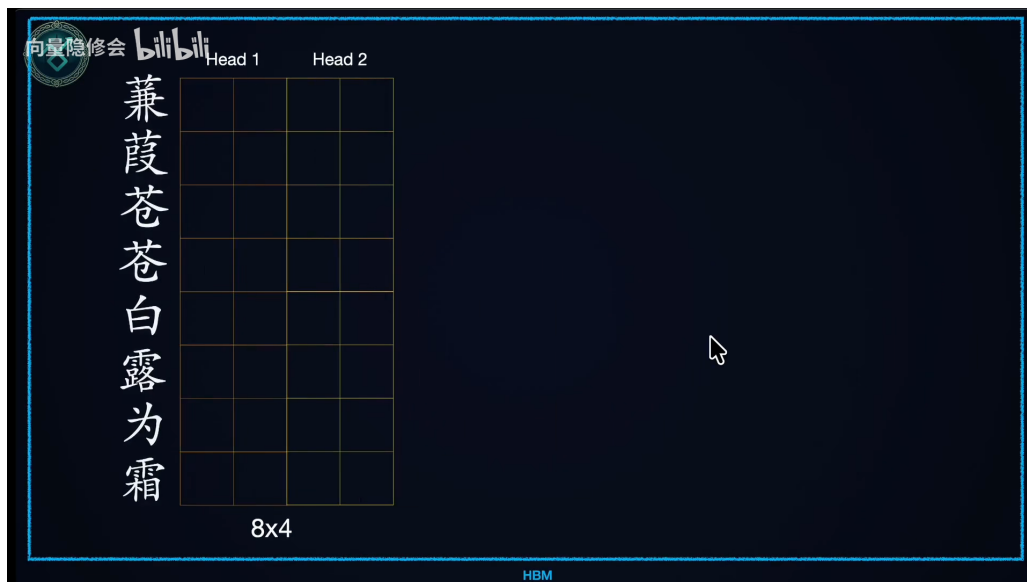


图 11: 两个头各自算出 8×2 的结果后拼接成 8×4 的矩阵 (Head 1 | Head 2)，暂存于 HBM。¹⁰

6.2 多头混合：输出投影 W_O

到这里还没结束。拼出来的 8×4 矩阵里，第一头和第二头的结果是泾渭分明、互不交流的。我们希望把两头信息混合，同时不改变矩阵形状（仍是 8×4 ）。这就需要有一个 4×4 的输出投影矩阵 W_O ：

$$\text{Output} = \underbrace{[\text{Head}_1 \mid \text{Head}_2]}_{8 \times 4} \cdot \underbrace{W_O}_{4 \times 4} = \underbrace{\text{最终结果}}_{8 \times 4}.$$

做一次矩阵乘 (Tensor Core)，就得到两头混合后的 8×4 最终自注意力结果。

6.3 自注意力计算流程总览

把整条流水线串起来，并标注每步的矩阵形状与所用计算核心：

¹⁰ 视频画面时间区间：00:17:31–00:17:40。

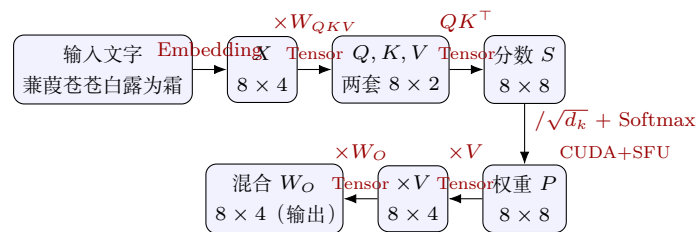


图 12: 自注意力在 GPU 上的计算流程总览 (笔记自绘): 每一步的矩阵形状 (本例 8 字 / 2 头 / 头维 2) 与所用计算核心。Tensor Core 扛起所有矩阵乘, CUDA Core/SFU 负责缩放与 Softmax。

6.4 本章小结

$P \times V$ (Tensor Core) 得到两头各 8×2 、拼成 8×4 ; 再经 4×4 的 W_O 混合, 得到最终 8×4 的自注意力输出。整条链路就是”搬进来—算—写回去”的不断重复。

7 总结与延伸: 藏在流程里的”内存墙”

7.1 作者的收尾: 问题出在哪

把整个过程回放一遍会发现一个刺眼的事实: 我们总是在反复地把数据从 HBM 读进 SM, 算出中间结果 (注意力分数矩阵、权重矩阵……) 后又写回 HBM, 下一步再读回来。而:

内存墙 (Memory Wall)

HBM 的读写速度, 远低于 SM 内 CUDA Core / Tensor Core 的计算速度。于是 SM 经常不是卡在”算不过来”, 而是卡在”等数据”——等着从 HBM 把数据搬进来、等着把结果写回去。计算单元大量时间在空转。这种被访存带宽拖住、而非被算力拖住的状态, 就是 **内存墙 / IO 瓶颈**。

这正是 FlashAttention 的起点

本讲的计算逻辑完全正确, 但它把每个中间矩阵 (S 、 P 等) 都老老实实写回了 HBM 又读回来。**FlashAttention 的核心思想**就是: 通过分块 (tiling) 和在线 Softmax, 让中间结果尽量留在快速的 SRAM 里、**不落盘到 HBM**, 从而绕过内存墙。本讲特意把”为什么 Softmax 要减最大值”留到下一讲——因为它正是”在线 Softmax”得以成立的关键。

7.2 笔者的延伸提炼

- **一条贯穿全篇的优化母题**: 把会被反复使用的数据, 留在更快、更靠近计算核心的存储层级里, 避免重复访问慢速的 HBM。无论是 W_{QKV} 只加载一次、还是 K_1/V_1 在缓存里被两个 SM 共享, 都是这一思想的体现; FlashAttention 把它推到极致。
- **并行有两个维度**: 在本例中是”多头”与”多 SM (序列分块)”的叉乘。理解这种切分方式, 是理解后续张量并行、序列并行的基础。
- **算子到硬件的映射**: 矩阵乘 $\rightarrow$ Tensor Core, 标量运算 $\rightarrow$ CUDA Core, 指数/对数 $\rightarrow$ SFU。看一个模型快不快, 既要看法力, 也要看它在存储层级间搬运了多少数据。

- 承上启下：本讲（朴素计算）→ FlashAttention（省显存 IO）→ KV Cache（推理省重复计算）→ PagedAttention（显存管理），构成了一条完整的”注意力在 GPU 上如何高效运行”的学习路径。

7.3 拓展阅读

- Vaswani et al., *Attention Is All You Need*, 2017——自注意力的原始定义。
- Dao et al., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, 2022。
- 关键词: *Tensor Core*、*GPU Memory Hierarchy*、*HBM vs. SRAM bandwidth*、*Online Softmax*。
- 视频原址: <https://www.bilibili.com/video/BV1zFFTziEdL> (“向量隐修会”注意力优化系列第一讲)。